ArduPilot Implementation of Optimal Control for Tilt Rotor UAV Landing

A thesis submitted in part fulfilment of the degree of
Bachelor of Engineering (Honours)

by
Lincoln Karavas
U7091679

Supervisor: Mr. Matthew Hampsey
Examiner: Prof. Robert Mahony



Australian National University

College of Engineering and Computer Science
The Australian National University

November 2026

This thesis contains no material which has been accepted for the award of any other degree or diploma in any university. To the best of the author's knowledge, it contains no material previously published or written by another person, except where due reference is made in the text.

Lincoln Karavas
6 November 2026

©Lincoln Karavas

Acknowledgements

I would like to thank my supervisors Matthew Hampsey, Andrew Tridgell and Robert Mahony for all of their guidance and time over the last year.

Abstract

This project puts forward an implementation of optimal control in ArduPilot to reduce the energy consumption of tilt rotor Unmanned Aerial Vehicles (UAVs) during landing by 61% compared to the current ArduPilot approach, with no hardware changes required. This is achieved through the use of a time-varying linear quadratic regulator control approach and a precomputed optimised trajectory. The goal of this project is to eventually expand this work to all powered-lift aircraft currently running ArduPilot. The other contribution of this work is the introduction of momentum drag into the ArduPilot simulation environment software in the loop (SITL), allowing users to simulate new manoeuvres utilising these effects or have more accurate simulation and flight planning.

Contents

Acknowledgements i

Abstract ii

List of Figures v

List of Tables vi

List of Tables vi

Nomenclature vii

1 Introduction 1

2 Literature Review 2

2.1 Powered Lift Aircraft and the Back-Transition . . . 2
2.2 Transition Control . . . 3
2.3 Optimal Control . . . 3
2.4 Post-Stall Manoeuvring and Perching . . . 4
2.5 Trajectory Tracking with Time-Varying LQR . . . 4
2.6 Rotor Aerodynamics in Non-Axial Flow . . . 5
2.7 Implementation on Open-Source Autopilots . . . 5
2.8 Summary and Research Gap . . . 6

3 Methodology 7

3.1 Aircraft Dynamical Model . . . 7
3.1.1 Aerodynamic Surface Forces . . . 8
3.1.2 Rotor Forces . . . 10
3.2 Trajectory Optimisation . . . 12
3.3 Control . . . 13
3.3.1 LQR . . . 13
3.3.1.1 LQR Gains . . . 13
3.3.2 Lateral Stabilisation . . . 14
3.4 Simulation Environment . . . 16
3.4.1 Rotor Model . . . 16
3.5 Safety Features . . . 16

4 Results and Analysis 17

4.1 Trajectory Result . . . 17
4.2 Control Results . . . 19
4.2.1 Landing Rejection . . . 25

5 Conclusions and Future Development **26**
5.1 Conclusion 26
5.2 Future Development 26
5.2.1 Validation 26
5.2.2 Model Expansion 26
5.2.3 Airframe types 26
5.2.4 System Identification 26

A Appendix A: The First **28**

Bibliography **29**

List of Figures

1.1 Arace Griffin Pro. 1

2.1 ArduPilot Landing Approach 2
2.2 Vortex Ring State Air Flow 3

3.1 Dynamical Model Diagram 7
3.2 Griffin Pro Wing Aerodynamic Coefficients 9
3.3 Griffin Pro Elevator Aerodynamic Coefficients at $\epsilon = 0$ 9
3.4 Griffin Pro Rotor Lift 11
3.5 Griffin Pro Rotor Drag 11
3.6 Longitudinal Control Block Diagram 13
3.7 Rotor Control Block Diagram 14
3.8 Lateral Control Block Diagram 14
3.9 LQR Re(λ_{max}) 15
3.10 LQR Rotor Gains 16

4.1 Trajectory Optimisation Solution 17
4.2 Trajectory Optimisation Solution States 18
4.3 Trajectory Optimisation Solution Control Inputs 18
4.4 Landing Approach Energy States 19
4.5 Simulation x and $\dot{x}$ 20
4.6 Simulation z and $\dot{z}$ 20
4.7 Simulation θ and $\dot{\theta}$ 20
4.8 Simulation y and $\dot{y}$ 20
4.9 Simulation ϕ and $\dot{\phi}$ 21
4.10 Simulation ψ and $\dot{\psi}$ 21
4.11 Simulation ω_{1-4} 22
4.12 Simulation ϵ and ξ 22
4.13 Headwind Step Response Peak in Hover 23
4.14 Crosswind Step Response Peak in Hover 24
4.15 Landing Approach Energy Consumption 24
4.16 Landing Approach Power 25
4.17 Altitude Floor Rejected Landing (15m) 25

List of Tables

3.1 Model States and Inputs 8
3.2 Aerodynamic Surface Coefficients 10
3.3 Rotor Parameters 10
3.4 Lateral Controller Gains 15

Nomenclature

UAV	Unmanned Aerial Vehicle
SITL	Software In the Loop
LQR	Linear Quadratic Regulator
TVLQR	Time Varying Linear Quadratic Regulator
MPC	Model Predictive Control
VTOL	Vertical Takeoff and Landing
STOVL	Short Takeoff and Vertical Landing
NLP	Non Linear Program
PD	Proportional Derivative
PID	Proportional Integral Derivative
SISO	Single Input Single Output
ESC	Electronic Speed Controller
PWM	Pulse Width Modulation

Introduction

This project puts forth a method and implementation for ArduPilot of optimal control for tilt rotor Unmanned Aerial Vehicles (UAV) landing transitions (specifically the Arace Griffin Pro) to reduce the energy consumption and landing period by 61% and 51% respectively compared to the stock ArduPilot approach. As the use of small and medium scale powered lift UAVs increases in commercial settings so too does the demand for greater endurance and efficiency. This is generally approached from a hardware perspective, through higher density batteries, more efficient propulsion or lighter and more efficient air frames. However this project attempts to solve this through exploiting the dynamics of the aircraft to reduce control effort, through a optimal control approach. Trajectory and control optimisations for UAVs is not a new area of research, however, this project provides an new and easier way to implement this for end users, as the control is integrated into ArduPilot, an existing and widespread approach to autonomous control of UAVs. This project specifically looks at the Arace Griffin Pro tilt rotor. This thesis starts by putting forth a longitudinal dynamical model of tilt rotor aircraft, by combining previous models of multirotor and fixed-wing aircraft. Followed by a method of trajectory optimisation, control and simulation.

Powered lift UAVs are a class of aircraft that have two separate flight modes. The first being fixed-wing flight like that of a conventional plane, the second being hover, similar to a multi-rotor. The combination of these two modes mean that the aircraft can takeoff and land without specialised infrastructure such as a runway, while being able to cruise in the more efficient fixed-wing flight mode, as the rotors in the fixed-wing mode only have to impart enough energy to balance the drag force, while in hover the rotors have to overcome the gravitational force of the entire airframe. Powered lift UAVs achieve this in different ways, some have separate propellers for hover and fixed-wing flight, some tilt their propellers from vertical to horizontal and some even tilt the entire wing with propellers statically mounted. These different airframe types all different approaches to transitioning between flight modes, but this project specifically looks at the Arace Griffin Pro a tilt rotor aircraft (Figure 1.1).



Figure 1.1: Arace Griffin Pro.

Literature Review

This chapter reviews the literature relevant to the energy optimised back-transition of a tilt rotor UAV. It begins with powered lift aircraft and the transition problem, then covers the control approaches that have been applied to it, followed by the trajectory optimisation methods used to generate transition manoeuvres. Post-stall manoeuvres is then reviewed as it is the closest analogue to the trajectory produced in this project, followed by trajectory tracking with time-varying LQR, the modelling of rotor forces in non-axial flow, and finally the implementation of controllers on open-source autopilots. The chapter concludes by identifying the gap this thesis addresses.

2.1 Powered Lift Aircraft and the Back-Transition

Powered Lift UAVs combine the vertical take off and landing capability of a multi rotor with the efficient forward flight of a fixed wing aircraft [13, 27]. The downside of this is the complicated flight mode achieved during the transition between the two. Ducard [13], provide a review of the different air frame types and applicable control architectures. They categorised all powered lift aircraft into, tail sitters; an aircraft that pitches to vertical to perform vertical take offs and landings, tilt rotors; where either all or a subset of the rotors tilt from vertical to forward, tilt wings; similar to tilt rotors however, the wing tilts with the rotor and quadplanes; that have separate propulsion systems for hover and fixed wing flight. The aircraft used in this project is the Griffin Pro which is a tilt rotor, where only the front two rotors of the quad tilt, and the model used is a modified version of that presented by Miller [23].

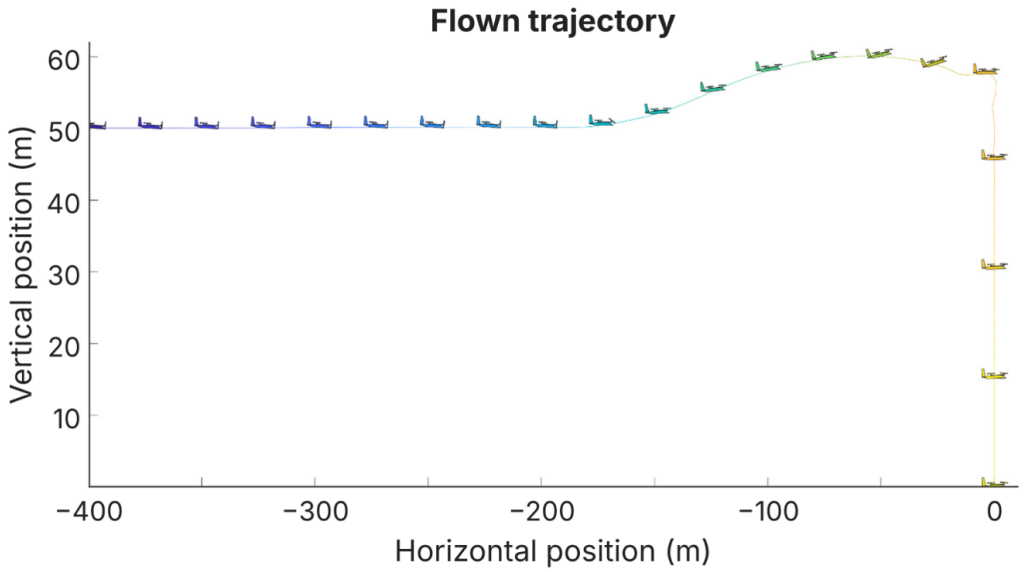


Figure 2.1: ArduPilot Landing Approach

The transition from hover to fixed wing flight has been extensively studied as there is a much greater risk for instability of the systems as the aircraft is in a lower energy state leading to unrecoverable stalls. The transition from fixed wing flight to hover is less studied as there is a obvious and stable solution of reconfiguring the aircraft for hover while coasting in fixed wing cruise. This is the approach used in current solutions such as ArduPilot. However this approach results in extensive time spent in hover, a substantially more power intensive flight mode compared to fixed wing flight (ref Figure 4.16), as the active rotors have to counteract the gravitational force rather than the passive wings. The obvious solution of descending quicker in hover to minimise time spent in that mode has its own issues, primarily entering the vortex ring state. A phenomenon where if a rotor powered aircraft descends at a comparable rate to the downwash velocity produced by the rotor, large vortices can form centered around the mid point of the rotor blade span and cause a dramatic loss in lift [10].

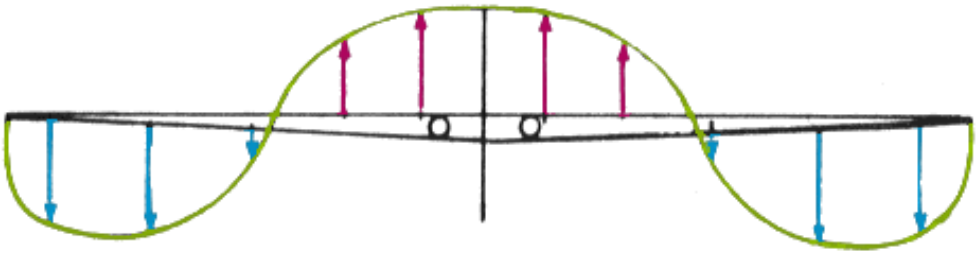


Figure 2.2: Vortex Ring State Air Flow

2.2 Transition Control

The simplest approach to transition control, and the one used by most open-source autopilots, is a scheduled transition, where the tilt angle and control transition are blended as functions of airspeed or time and the attitude is stabilised by PID loops [5]. This approach is robust and simple to tune, but it is not optimal with respect to energy consumption.

Model-based controllers have already been employed to optimise these transitions. Allenspach and Ducard [1] applied model predictive control (MPC) to a tilt rotor UAV, and achieved one unified controller for all stages of flight. Ducard and Allenspach [13] propose that this single controller approach should be the direction the field moves in, along with the need to manage control complexity against the computing resources available on board, as this is a significant constraint for MPC, which requires an optimisation problem to be solved in real time on board the aircraft's flight controller, and is a primary motivation for the precomputed controller used in this project.

Learning based approaches have also been applied. Fu et al. [15] used safe reinforcement learning for the back-transition of a ducted fan tail-sitter, minimising altitude change and transition time subject to a vertical velocity constraint, and showed that the learned trajectory closely matched the optimal trajectory obtained with GPOPS-II.

2.3 Optimal Control

Trajectory optimisation refers to methods that find the control inputs which take a dynamic system between two states while minimising a cost function and satisfying the associated constraints. Direct

methods transcribe this infinite dimensional problem into a finite dimensional nonlinear program (NLP) by discretising the states and controls through time. Hargraves and Paris [16] introduced direct collocation, in which the system dynamics are enforced as equality constraints between discrete points, and Betts [8] summarises the numerical methods used to solve such problems.

Kelly [17] provides a method on direct collocation, covering trapezoidal and Hermite–Simpson collocation, initialisation, and mesh refinement, and released the MATLAB library OptimTraj [18] that implements these methods; OptimTraj is the library used in this project. Kelly notes that the NLPs produced by transcription are generally non-convex, so the solver returns a locally optimal solution which depends on the initial guess. The straight line initialisation and successive mesh refinement described in Section 3.2 are intended to manage this.

Trajectory optimisation has been applied to the transition problem by several authors. Naldi and Marconi [25] formulated minimum-time and minimum-energy transition manoeuvres for a class of V/STOL aircraft as mixed-integer nonlinear programs. Zhao et al. [29] computed a transition corridor (map of feasible states based of tilt angle and velocity) for a tilt rotor UAV and solved the transition strategy as a constrained optimal control problem, with a cost function built on transition time, attitude stability and control continuity. Deng et al. [12] extended static transition corridor to a dynamic one allowing for the aircraft to be in not be in equilibrium as well as adding angle of attack to form a three dimensional space of feasible states. Both of these approaches support modelling exclusively the longitudinal dynamics of the aircraft due to the symmetry of the airframes. The other common factor of these two papers is the exclusion of stalled states from the trajectory, as they don't model post stall dynamics, and they lie outside the feasible states for the aircraft.

2.4 Post-Stall Manoeuvring and Perching

The optimal trajectory found in this project dives to convert potential energy into kinetic energy, then pulls up into a stall, using the drag at high angle of attack to dissipate the majority of the aircraft's kinetic energy. This manoeuvre resembles that of perching birds, which has been studied in the context of fixed wing UAVs.

Cory and Tedrake [11] demonstrated a fixed wing glider performing a perching manoeuvre using a model identified from motion capture data and a trajectory optimised for the post-stall regime, stabilised with a time-varying LQR. Moore, Cory and Tedrake [24] extended this work using LQR-Trees, a feedback motion planning method built on time-varying LQR around nominal trajectories, and achieved a 95% perching success rate over 147 flights on real hardware across a wide range of initial speeds. Their results show that post-stall manoeuvres can be executed reliably with a simple airframe and a model based controller, without complex dynamic wings, at least in the absence of significant wind gusts.

These works establish that deliberately stalling an aircraft is a viable way to dissipate energy, and that time-varying LQR about an optimised trajectory is a suitable way to stabilise it. The tilt rotor in this project has an advantage over the glider in that the rotors provide control authority that is independent of airspeed, which is used to stabilise the stall in the final stage of the approach.

2.5 Trajectory Tracking with Time-Varying LQR

The linear quadratic regulator (LQR) minimises a squared cost on state error and control effort, with the optimal gain given by the solution of a Riccati equation [2]. For a time-varying system linearised

about a trajectory over a finite horizon, the gain is found by integrating the differential Riccati equation backwards in time from a terminal condition, which produces a time-varying gain matrix [2, 28]. As the horizon is finite, the gains reduce towards the end of the trajectory as the remaining cost of a state error becomes small relative to the control cost. This explains the reduction in gain seen near the end of the approach in Figure 3.10. The weighting matrices are commonly chosen using Bryson's rule [9], where each diagonal weight is the inverse square of the maximum acceptable value of the corresponding state or input.

Time-varying LQR (TVLQR) should be distinguished from classical gain scheduling, where independent controllers are designed at a set of frozen operating points and interpolated as a function of a scheduling variable such as airspeed [26]. Rugh and Shamma [26] note that stability at each frozen operating point does not in general guarantee stability of the scheduled system, unless the scheduling variable changes slowly. The same caution applies to the frozen time eigenvalues of a time-varying closed loop system, so the stability of the controller in this project is also assessed along the trajectory as a whole rather than from the frozen time eigenvalues alone.

TVLQR is well suited to embedded implementation because all of the computation is done offline, and the onboard controller only needs to look up a gain and compute a matrix vector product each cycle. This is in contrast to MPC, and is the reason it was selected for this project.

2.6 Rotor Aerodynamics in Non-Axial Flow

When a rotor operates with a component of the freestream perpendicular to its axis of rotation, the advancing blade sees a higher airspeed than the retreating blade, producing an in-plane force opposing the motion (the H-force) and blade flapping, as described by momentum and blade element theory [20]. Martin and Salaün [22] showed that this rotor drag is significant even on small multirotors at low speed. Mahony, Kumar and Corke [21] include blade flapping and induced drag in their tutorial model of multirotor dynamics, and Bangura and Mahony [7] used momentum theory to model rotor thrust for multirotor control. Faessler et al. [14] showed that modelling rotor drag explicitly, rather than treating it as an unknown disturbance, significantly reduces the trajectory tracking error of a quadrotor.

The optimal trajectory in this project uses the rotors in this way, with the aircraft's velocity perpendicular to the rotor axes during the stall, so that rotor drag contributes to dissipating the kinetic energy. The public release of the ArduPilot simulator does not model this force, which motivated the rotor model developed in Section 3.1.2.

2.7 Implementation on Open-Source Autopilots

ArduPilot is an open-source autopilot that supports tilt rotor and other powered lift VTOL aircraft through its QuadPlane firmware [5]. It provides a Lua scripting environment that allows custom scripts to read the vehicle state and override the outputs to the servos and motors without modifying the firmware [4], and a software in the loop (SITL) simulator that runs the unmodified flight code against a simulated aircraft [6]. Together these allow a controller to be deployed on an existing flight controller of a production aircraft and tested in simulation first, which is the approach taken in this project. The limited memory and processing available to the Lua environment are the main constraints on the controller, and motivate both the choice of TVLQR and the reduction in the number of stored gain matrices described in Section 3.3.1.1.

The accuracy of any model based controller depends on how closely the aerodynamic model matches the real aircraft. The aerodynamic coefficients of the model in this project were re-estimated from flight logs of the Griffin Pro, following the established approach of aircraft system identification from flight data [19].

2.8 Summary and Research Gap

The literature shows that the transition of powered lift VTOL aircraft has been approached through scheduled PID control, model predictive control, reinforcement learning, and trajectory optimisation within a transition corridor. The majority of this work focuses on the forward transition, constrains the aircraft to near trimmed flight, and optimises for smoothness, time and attitude rather than energy. Separately, the perching literature shows that post-stall manoeuvres can be stabilised reliably with time-varying LQR, and the multirotor literature shows that rotor drag in non-axial flow is a significant force that can be modelled and exploited.

No work was found that combines these to find an energy optimal back-transition for a tilt rotor that uses both the post-stall drag of the wing and the in-plane drag of the rotors, and implements it on an unmodified production flight controller. This thesis addresses that gap by optimising the back-transition of the Griffin Pro for minimum rotor energy, tracking it with a TVLQR implemented in ArduPilot's Lua environment.

Methodology

3.1 Aircraft Dynamical Model

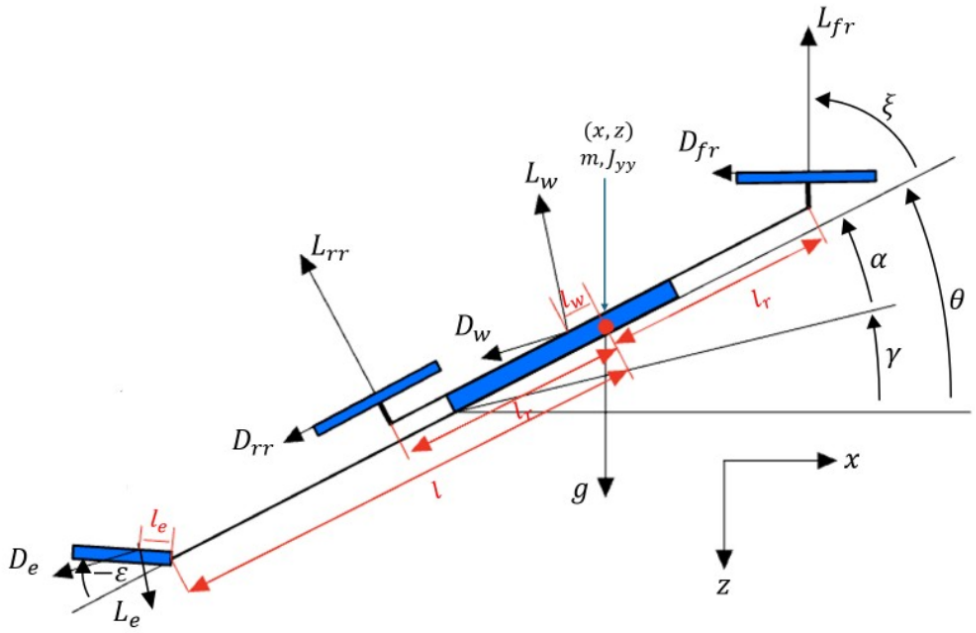


Figure 3.1: Dynamical Model Diagram

The Model used in this project is based of that presented in Miller's thesis [23]. As such the model only captures the longitudinal dynamics of the aircraft, this simplification was made as all of the behaviours being optimised are only present in the longitudinal case, while the lateral is being stabilised to trim level flight. The simplification to a two dimensional model means that the state space X can be simplified from $\mathbb{R}^{12}$ to $\mathbb{R}^6$, the input space (U) can also be simplified resulting in the spaces shown in (3.1)

$$X = \begin{bmatrix} x & z & \theta & \dot{x} & \dot{z} & \dot{\theta} \end{bmatrix}^T, \qquad U = \begin{bmatrix} \omega_f & \omega_r & \xi & \epsilon \end{bmatrix}^T \tag{3.1}$$

where

State		Input	
x	e_1 position (m)	ω_f	front rotor speed (rpm)
z	e_2 position (m)	ω_r	rear rotor speed (rpm)
θ	pitch (rad)	ξ	front rotor tilt (rad)
$\dot{x}$	e_1 velocity (m/s)	ϵ	elevator deflection (rad)
$\dot{z}$	e_2 velocity (m/s)		
$\dot{\theta}$	pitch rate (rad/s)		

Table 3.1: Model States and Inputs

The dynamics of the system are shown in (3.2) & (3.3), which are an application of Newton's second law of motion rearranged.

$$\begin{bmatrix} \ddot{x} \\ \ddot{z} \end{bmatrix} = \frac{1}{m}(R(\theta-\alpha)(F_w+F_e)+R(\theta+\xi)F_{fr}+R(\theta)F_{rr}+mg) \tag{3.2}$$

$$\ddot{\theta} = \frac{1}{J_{yy}}(l_w R(-\alpha)F_{w_\perp}+(lcos(\epsilon)+l_e)R(-\alpha)F_{e_\perp}-l_r R(\theta+\xi)F_{fr_\perp}+l_r R(\theta)F_{rr_\perp}) \tag{3.3}$$

where $R(\theta)$ is the standard two dimensional rotation matrix.

3.1.1 Aerodynamic Surface Forces

The aerodynamic surface (wing & elevator) forces are modelled using a blended coefficients model to describe post-stall behaviour, as the standard approach of a linear model would mislead the trajectory optimisation when the aircraft would be in stall. The blended model approach utilises the standard linear airfoil coefficient at angles of attack less than stall and smoothly transitions to a flat-plate model post-stall via a sigmoid function.

$$L_x = \frac{1}{2}s_x\rho|v_x^2|C_{l_x}(\alpha_x), \quad D_x = \frac{1}{2}s_x\rho|v_x^2|C_{d_x}(\alpha_x) \tag{3.4}$$

In the following set of equations ((3.4)- (3.9)) x represents a specific aerodynamic surface of the aircraft. The total coefficients (3.5) show how this blending occurs.

$$\begin{aligned} C_{l_x}(\alpha_x) &= (1-\sigma_x)C_{l_{x_{airfoil}}}(\alpha_x)+\sigma_x C_{l_{flatplate}}(\alpha_x) \\ C_{d_x}(\alpha_x) &= (1-\sigma_x)C_{d_{x_{airfoil}}}(\alpha_x)+\sigma_x C_{d_{flatplate}}(\alpha_x) \end{aligned} \tag{3.5}$$

$$C_{l_{x_{airfoil}}}(\alpha_x) = C_{l0_x}+C_{l\alpha_x}(\alpha_x), \quad C_{d_{x_{airfoil}}}(\alpha_x) = C_{d0}+C_{d\alpha_x}(\alpha_x^2), \tag{3.6}$$

$$C_{l_{flatplate}}(\alpha) = 2sin(\alpha)cos(\alpha), \quad C_{d_{flatplate}}(\alpha) = 2sin^2(\alpha) \tag{3.7}$$

This model for the elevator accounts for the variation in force due to the elevator deflection by applying a modifier to the elevator angle of attack proportional to the deflection. The value of this proportional gain (τ) was fit log data from real world Griffin Pro flights.

$$\alpha_w = tan^{-1}\frac{\dot{z}}{\dot{x}} \qquad \alpha_e = \alpha_w+\epsilon\tau \tag{3.8}$$

$$\sigma_x(\alpha) = \frac{1+e^{-M_x(\alpha-t_{l_x})}+e^{M_x(\alpha-t_{u_x})}}{(1+e^{-M_x(\alpha-t_{l_x})})(1+e^{M_x(\alpha-t_{u_x})})} \tag{3.9}$$

The Resulting Coefficients of the wing can be seen in Figure 3.2 where we have a continuous function throughout $[-\pi,\pi]$

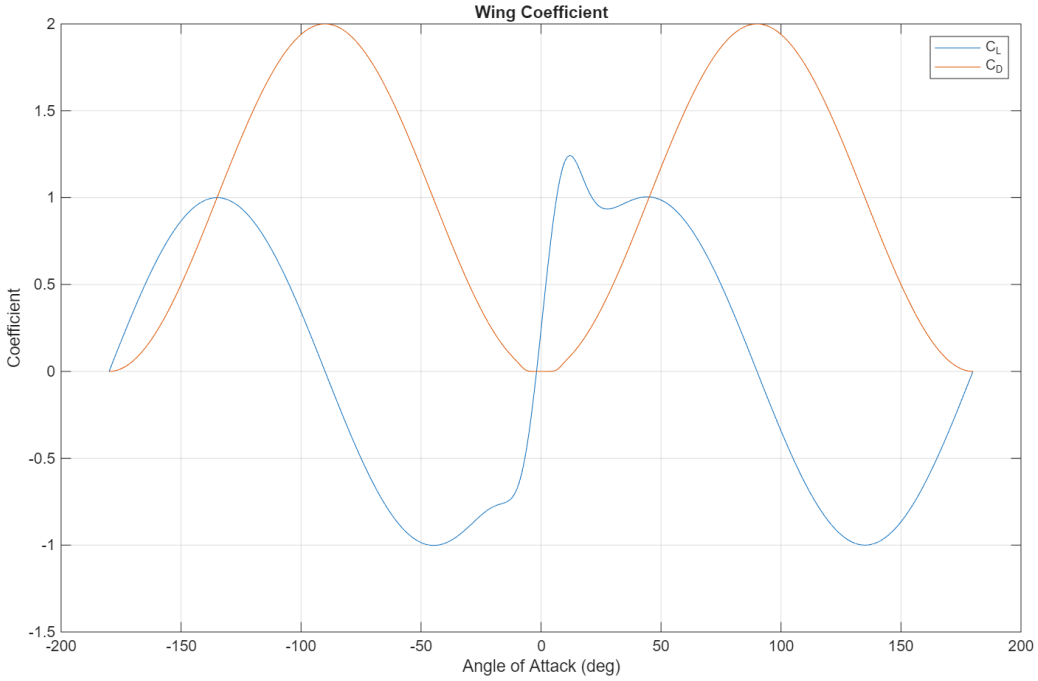


Figure 3.2: Griffin Pro Wing Aerodynamic Coefficients

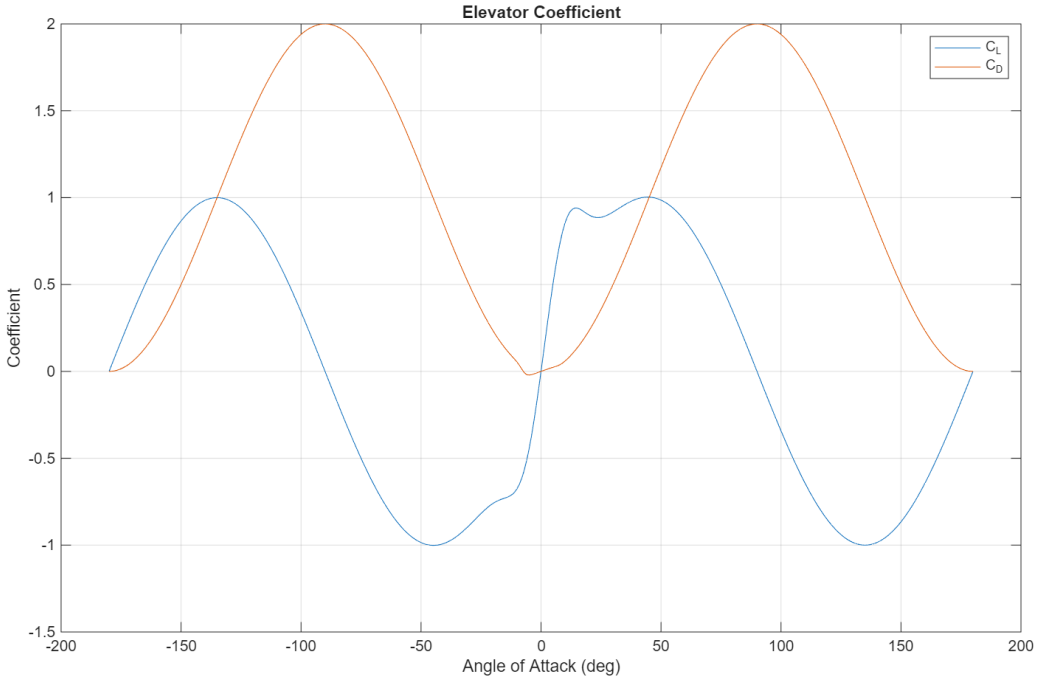


Figure 3.3: Griffin Pro Elevator Aerodynamic Coefficients at $\epsilon = 0$

Coefficient	Value	Coefficient	Value
C_{l0_w}	1.9164	$C_{l\alpha_w}$	11.6632
C_{l0_e}	0	$C_{l\alpha_e}$	6.8755
C_{d0_w}	0.075	$C_{d\alpha_w}$	4.015
C_{d0_e}	0 .01	$C_{d\alpha_e}$	1.7968
t_{l_w}	-0.18	t_{u_w}	0.28
t_{l_e}	-0.28	t_{u_e}	0.28
M_w	1	M_e	26
τ	2.7		

Table 3.2: Aerodynamic Surface Coefficients

3.1.2 Rotor Forces

The trajectory in section 4.1 utilises the non axial drag component the rotors produce when the freestream velocity has a perpendicular component to the rotor axle to dissipate kinetic energy from the aircraft, to model this behaviour the following equations were employed.

$$L_{front} = \frac{1}{2} N_b \rho c R_r^3 \omega_{front}^2 C_T, \qquad D_{front} = \frac{L_{front}(\dot{x}_{front})}{R_r \omega_{front}} \tag{3.10}$$

$$L_{rear} = \frac{1}{2} N_b \rho c R_r^3 \omega_{rear}^2 C_T, \qquad D_{rear} = \frac{L_{rear}(\dot{x}_{rear})}{R_r \omega_{rear}} \tag{3.11}$$

where N_b is the number of blades of the rotor, ρ is air density,c is the average chord of the airfoil, R_r is the radius of the rotor and C_T is the coefficient of thrust, it was estimated for hover by solving equation (3.13) which is the lift of the front and rear rotors equated to the weight force of the aircraft with the values from a real world flight log substituted for m, ρ, ω

$$\dot{X}_{front} = R_{IR}(\theta + \xi) X_{inertial}, \qquad \dot{X}_{rear} = R_{IR}(\theta + \xi) X_{inertial}, \qquad X = \begin{bmatrix} x \\ z \end{bmatrix} \tag{3.12}$$

$$C_T = \frac{2mg}{N_b \rho c R_r^3 (\omega_{front} + \omega_{rear})} \tag{3.13}$$

The parameters used for the Griffin Pro are shown in 3.3 and result in the lift and drag curves (3.4 - 3.5). The velocity referred to in 3.5 is the non axial component.

Coefficient	Value	Coefficient	Value
N_b	2	ρ	1.224
c	0.025	R_r	0.175
C_T	0.008404		

Table 3.3: Rotor Parameters

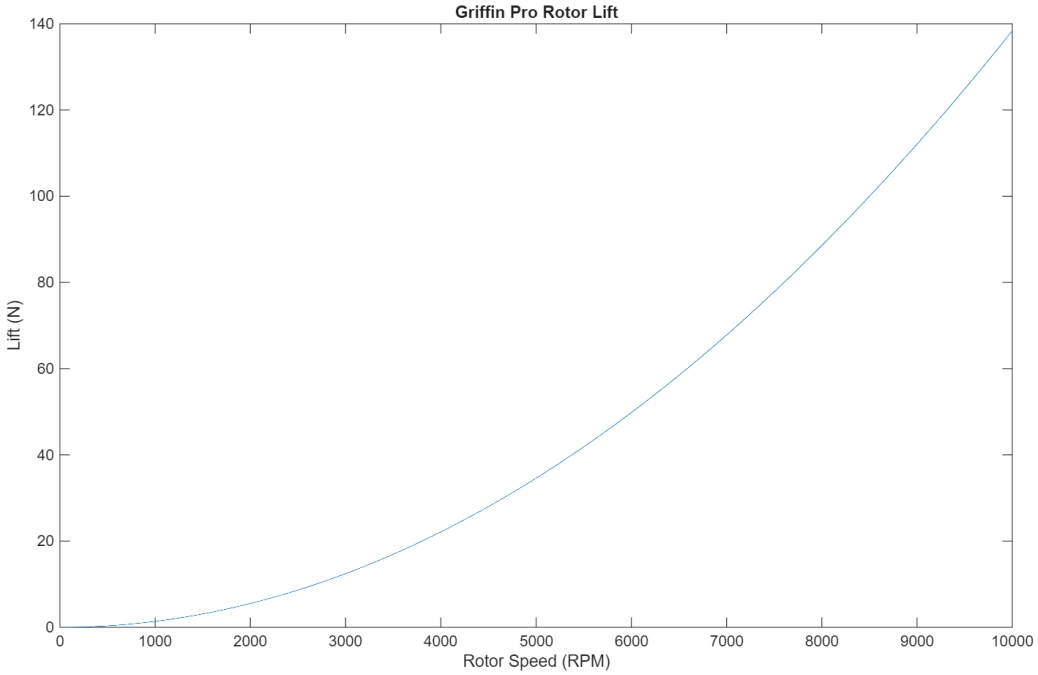


Figure 3.4: Griffin Pro Rotor Lift

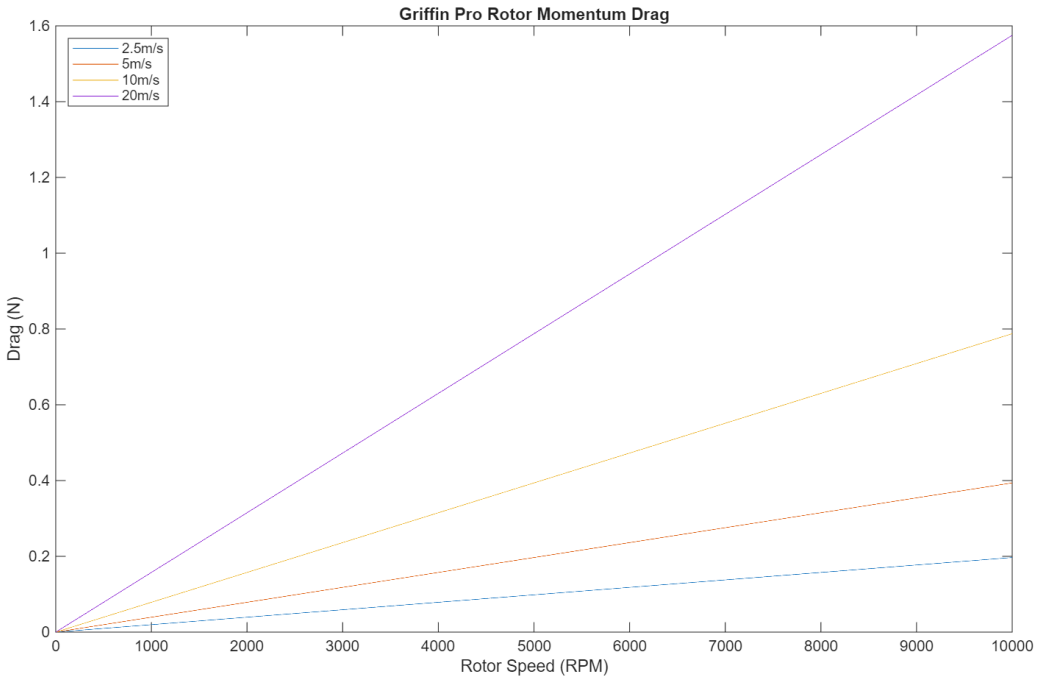


Figure 3.5: Griffin Pro Rotor Drag

3.2 Trajectory Optimisation

The MATLAB library OptimTraj [18] was used to solve the optimisation problem for the transition trajectory. OptimTraj utilises MATLAB's fmincon solver.

The constraints for the trajectory optimisation were as follows.

$$Y_{initial} = \begin{bmatrix} -400 & -50 & -0.0623 & 22 & 0 & 0 \end{bmatrix}^T, \qquad Y_{final} = \begin{bmatrix} 0 & -10 & 0 & 0 & 0 & 0 \end{bmatrix}^T \tag{3.14}$$

$$Y_{lower} = \begin{bmatrix} -\infty & -\infty & -\pi/10 & -\infty & -\infty & -\pi/12 \end{bmatrix}^T, \qquad Y_{upper} = \begin{bmatrix} \infty & -10 & \pi/3 & 22 & \infty & \pi/12 \end{bmatrix}^T \tag{3.15}$$

$$U_{lower} = \begin{bmatrix} 0 & 0 & 0 & -\pi/4.75 \end{bmatrix}^T, \qquad U_{upper} = \begin{bmatrix} 8500 & 8500 & \pi/2 & \pi/4.75 \end{bmatrix}^T \tag{3.16}$$

$$11\pi/90 - \alpha >= 0, \qquad \alpha - \pi/18 >= 0 \tag{3.17}$$

The control constraints are determined by the capabilities of the aircraft, with the exception of the tilt angle (ξ) a 15% reduction to the upper limit was applied to ensure there was adequate control margin. Y_{final} is set to a stationary hover 10m above the ground as to allow the stock ArduPilot to take over for the touch down and complete its disarming protocols uninterrupted, this height can also be altered along the state upper constraint for altitude to set a minimum height that the pilot feels is appropriate. The final constraint was applied to the angle of attack of the airframe, to limit aggressiveness of the stalls the aircraft could perform, this too can be configured to a value the pilot deems appropriate.

The initialisation point of the optimisation was a straight line trajectory with the inputs at zero for all time.

$$Y_{initialisation} = \begin{bmatrix} Y_{initial}, & Y_{final} \end{bmatrix}, U_{initialisation} = 0^{\mathbb{R}^{4\times 2}} \tag{3.18}$$

The optimisation mesh would initially be calculated with 0.5s timesteps (Δt), and time would be a free variable. After the initial solution had converged, the total time would be rounded to the nearest 0.5s and be set as a constraint and the timesteps would be reduced to 0.25s, with the initial solution replacing the previous initialisations for state and control input. This process was repeated until a time step of 0.1s was achieved.

The goal of this optimisation is to find the most energy efficient landing transition trajectory, as such the cost function was set to the total energy consumed during the transition phase, with the assumption that elevator and tilt inputs would cause marginal differences the cost function was therefore simplified to be the energy the rotors require throughout the trajectory.

$$J_{trajectory} = \sum_{i=0}^{\frac{t_f}{\Delta t}} 2(P_{front_i} + P_{rear_i})\Delta t \tag{3.19}$$

The power estimate shown in 3.19 was derived using the method shown in [23].This method can be summarised as solving the set of equations 3.20 for P, by substituting the lift and drag values from 3.10 & 3.11.

$$\begin{aligned} L &= 2\rho\pi R_r^2 v_z^i \sqrt{(v_x^i + \dot{x}_{rotor})^2 + (v_z^i - \dot{z}_{rotor})^2}, \\ D &= 2\rho\pi R_r^2 v_x^i \sqrt{(v_x^i + \dot{x}_{rotor})^2 + (v_z^i - \dot{z}_{rotor})^2}, \\ P &= L(v_z^i - \dot{z}_{rotor}) + D(v_x^i + \dot{x}_{rotor}) \end{aligned} \tag{3.20}$$

3.3 Control

The control of the aircraft is achieved through a time varying linear quadratic regulator (LQR), utilising the optimisation solution as the feedforward term for the longitudinal control and a PID architecture for the lateral stabilisation. LQR was chosen over the other approaches mentioned in section 2.2 for its lower computational requirement, in order to meet the goal of having this controller run on existing hardware.

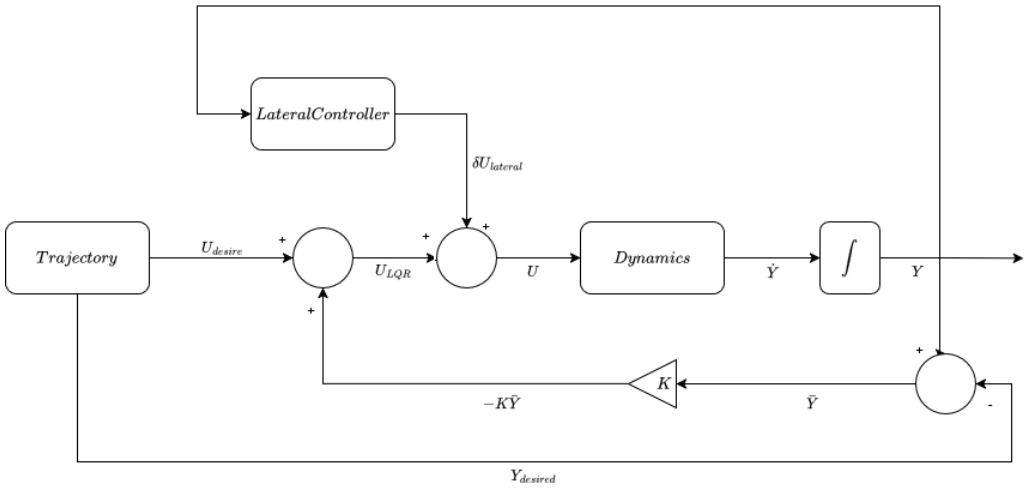


Figure 3.6: Longitudinal Control Block Diagram

The choice of LQR for the longitudinal dynamics was driven by the desire for an efficient control architecture with respect to the goal of this project minimising control energy and to minimise the computation required in real time on the flight controller, as the gains can be precomputed.

3.3.1 LQR

The LQR controller only controls the longitudinal dynamics as shown in section 3.1, therefore $Y_d \in \mathbb{R}^{6\times\frac{t_f}{\Delta t}}$ and $U_d \in \mathbb{R}^{4\times\frac{t_f}{\Delta t}}$ this is then linearly interpolated to produce $y_d(t) \in \mathbb{R}^6$ and $u_d(t) \in \mathbb{R}^4$. with these continuous vectors equation (3.21) can be formed, where $\bar{y}(t) = y(t) - y_d(t)$ and $\bar{u}(t) = u(t) - u_d(t)$

$$\dot{\bar{y}}(t) = f(\bar{y}(t), \bar{u}(t), t) \tag{3.21}$$

As the dynamics of the system are non-linear it was linearised using the Jacobians: $A(t) = \frac{\partial f(t)}{\partial y(t)}$, $B(t) = \frac{\partial f(t)}{\partial u(t)}$ to form equation (3.22), with a timestep of $1 \times 10^{-4}s$

$$\dot{\bar{y}}(t) = A(t)\bar{y}(t) + B(t)\bar{u}(t) \tag{3.22}$$

3.3.1.1 LQR Gains

The LQR Controller calculates the gains by solving the following cost function:

$$J_{LQR} = \int_0^{t_f} \bar{y}^T(t)Q\bar{y}(t) + \bar{u}^T(t)R\bar{u}(t)\,dt \tag{3.23}$$

where $Q \in \mathbb{R}^{6\times6}$ is the state error cost matrix and $R \in \mathbb{R}^{4\times4}$ is the control cost matrix. Both were set using Bryson's rule, stated below.

$$Q_{ii} = \frac{1}{\bar{y}^2_{i_{max}}}, R_{ii} = \frac{1}{\bar{u}^2_{i_{max}}} \tag{3.24}$$

resulting in the following values for the Griffin Pro: $Q = diag(4, 4, 14.5903, 1.111 \times 10^{-3}, 25, 8.2070)$ and $R = diag(4 \times 10^{-6}, 4 \times 10^{-6}, 1.3131 \times 10^{4}, 1.4590 \times 10^{3})$. With Q and R selected the differential Riccati equations can be solved moving back from the terminal point of the trajectory assuming zero final state and input error, to find the gain matrices.

$$-\dot{S}(t) = A^T(t)S(t) + S(t)A(t) - S(t)B(t)R^{-1}B^T(t)S(t) + Q \tag{3.25}$$

$$K(t) = R^{-1}B^T(t)S(t) \tag{3.26}$$

At this point $K \in \mathbb{R}^{4\times 6t_f 10000}$ is this much larger than the available memory on most flight controllers and therefore every $10,000^{th}$ term is kept and the rest were discarded forming a new matrix. This is the extent of the precomputation achieved, next $Y_d \in \mathbb{R}^{6\times t_f \Delta t}$, $U_d \in \mathbb{R}^{4\times t_f \Delta t}$, and $K_{reduced} \in \mathbb{R}^{4\times 6t_f}$ were imported into the ArduPilot Lua scripting environment.

Within the Lua script the control inputs are converted from their native units (Table 3.1) to μs which represents the pulse duration in the aircraft ESC. This utilises the ArduPilot function *"SRV_Channels:setoutput_pwm_chan_timeout"*. The elevator and tilt servos are converted by a linear map, however, the motor rpm to μs uses a linear map followed by a SISO PD controller on the rotor rpm feedback from the ESC.

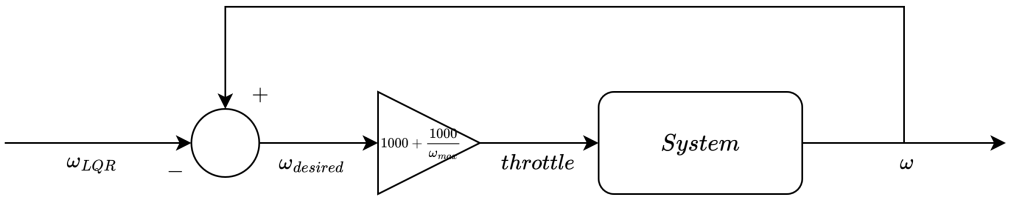


Figure 3.7: Rotor Control Block Diagram

3.3.2 Lateral Stabilisation

As the LQR models is limited to the longitudinal dynamics the lateral stabilisation is a non-optimal series of PD controllers, encompassing roll (ϕ), yaw (ψ) and y axis position, controlled through modifiers to the LQR control inputs for the motors and tilt servos, as shown in figure 3.8.

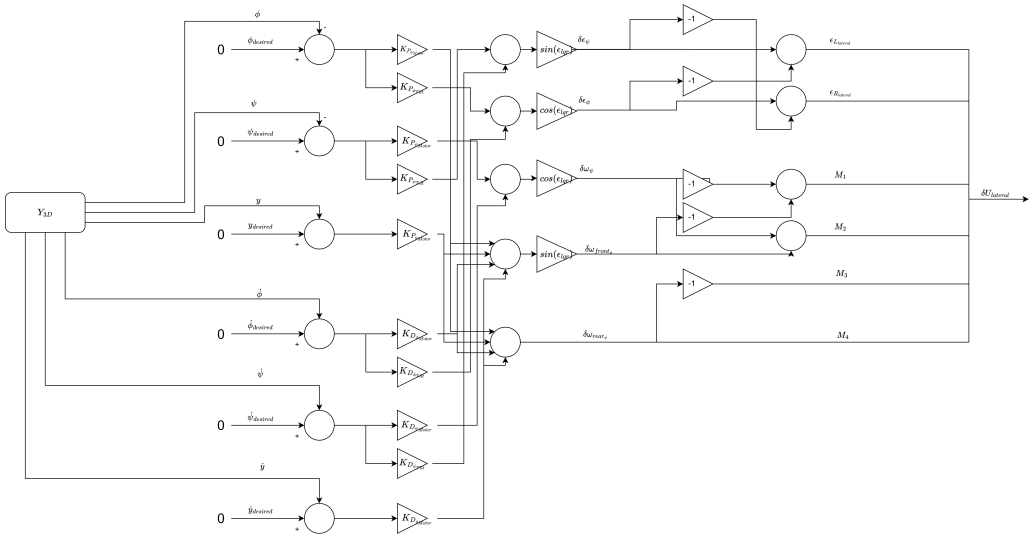


Figure 3.8: Lateral Control Block Diagram

The gains for these PD controllers were tuned with the aid of a large language model agent (Claude [3]) running software in the loop (SITL) runs until the state errors were below the required threshold: $\bar{Y}_{3D} = [0.1745, 0.1745, 1, 0.0524, 0.0524, 0.5]^T|$ resulting in the gains below.

$$a = 0.2\Delta t_{lua} + 1 \tag{3.27}$$

Gain	Value	Unit	Gain	Value	Unit
$K_{P_{\phi_{Motor}}}$	$\frac{300}{a}$	$rad- > \mu s$	$K_{D_{\phi_{Motor}}}$	$\frac{50}{a}$	$rad/s- > \mu s$
$K_{P_{\psi_{Motor}}}$	$\frac{600}{a}$	$rad- > \mu s$	$K_{D_{\psi_{Motor}}}$	$\frac{100}{a}$	$rad/s- > \mu s$
$K_{P_{y_{Motor}}}$	$\frac{60}{a}$	$m- > \mu s$	$K_{D_{\phi_{Motor}}}$	$\frac{120}{a}$	$m/s- > \mu s$
$K_{P_{\phi_\xi}}$	$\frac{0.2}{a}$	$rad- > \mu s$	$K_{D_{\phi_\xi}}$	$\frac{0.03}{a}$	$rad/s- > \mu s$
$K_{P_{\phi_\xi}}$	$\frac{0.3}{a}$	$rad- > \mu s$	$K_{D_{\phi_\xi}}$	$\frac{0.05}{a}$	$rad/s- > \mu s$

Table 3.4: Lateral Controller Gains

where Δt_{lua} is the time step between each cycle of the Lua script. Each of the five PD controllers is individually clamped, the max tilt alteration is 15 deg and $200\mu s$ for the PWM signals. The layering of the LQR and lateral controllers means that the stability of the system is not guaranteed, as such the poles of the full system were calculated. Figure 3.9 shows the maximum real value of the poles throughout the landing approach.

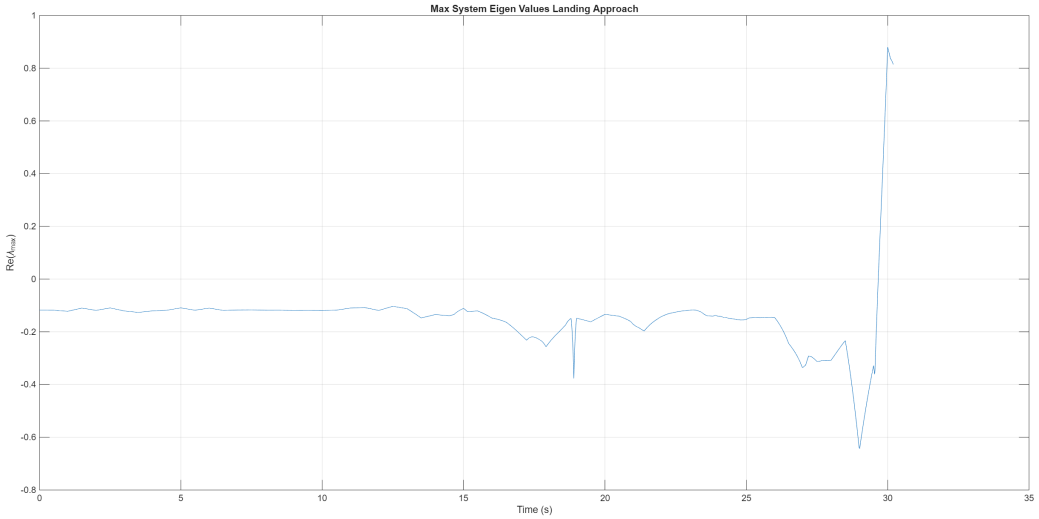


Figure 3.9: LQR Re(λ_{max})

The max real pole is negative throughout the approach ensuring stability until $t \approx 29.66s$. This is due to the LQR cost function minimising both state error and input cost, meaning that the LQR gains wind-down as the system approaches the time horizon because the error doesn't grow enough before ending to warrant the cost incurred by the rotors.

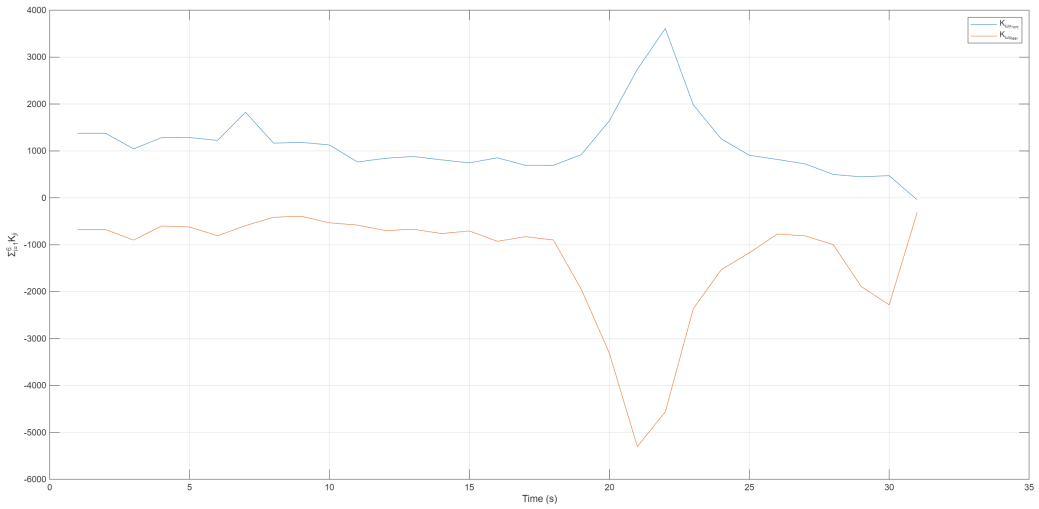


Figure 3.10: LQR Rotor Gains

3.4 Simulation Environment

3.4.1 Rotor Model

The public release of ArduPilot's simulation environment doesn't model the non-axial forces of the rotors and therefore I have modified the simulator to model the rotors utilising momentum theory. These changes to the rotor model work with any controller or aircraft running ArduPilot and thus other may wish to use it in their own simulations.

The implementations of these changes follow the same approach as those shown in section 3.1.2, with a variation in how the velocity is the drag term is calculated to work in the three dimensional simulation.

$$\dot{X}_{front} = R_{IR}(\phi, \theta + \xi - \frac{\pi}{2}, \psi), \qquad \dot{X}_{rear} = R_{IR}(\phi, \theta + \xi, \psi), \qquad X = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \tag{3.28}$$

3.5 Safety Features

The Altitude floor is a minimum altitude that if breached will cease the control input overrides of the Lua script, returning control to the stock ArduPilot controller, which brings the aircraft to a hover and then traverses as a multirotor to the landing point before hovering down to land. Another method for rejecting the landing sequence is if the norm of the state error surpasses a preset limit. This triggers in two cases, the first is the aircraft failing to track the trajectory, the second being a mismatch in the initial condition for example if the trajectory had not been updated and was made for a different altitude approach. This will also return to ArduPilot controller and behave as described above.

Results and Analysis

4.1 Trajectory Result

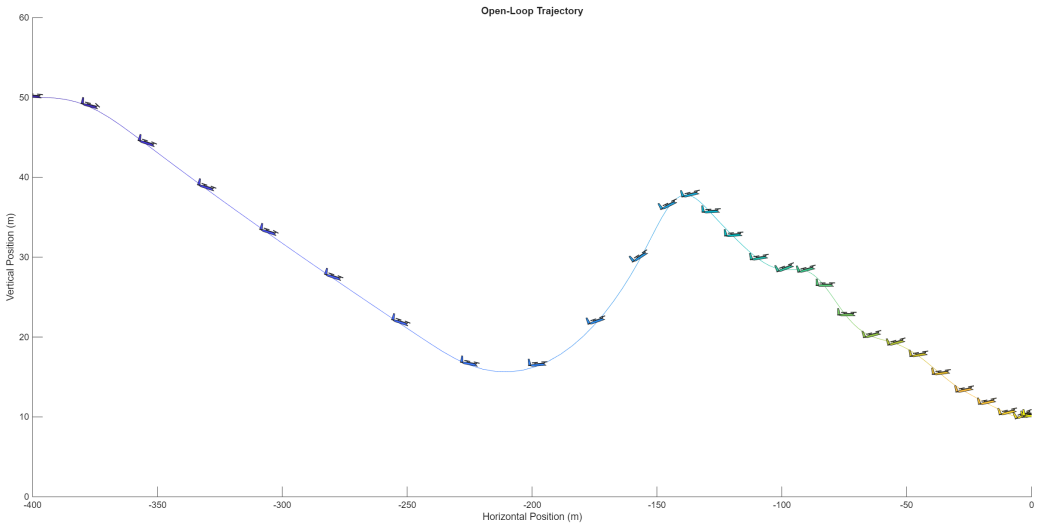


Figure 4.1: Trajectory Optimisation Solution

The solution of the trajectory optimisation can be found in 4.2 & 4.3, where you can see the aircraft initially enters a dive, riding the negative pitch constraint ($-20\,deg$) until an altitude of $17\,m$. At which point the aircraft pulls up, briefly hitting the positive pitch rate constraint ($15\,deg/s$), until stall is achieved, which is subsequently held till the aircraft reaches the termination condition. This results in a total energy consumption of $4536\,J$ (This can't be compared directly with the results in 4.15 as this value doesn't include the energy associated with the final hover phase as that is not part of the trajectory optimisation). The inputs for the optimised trajectory (4.3) remained bounded throughout the approach and are absent of any oscillatory behaviour, providing a smooth feedforward term for the controller to track.

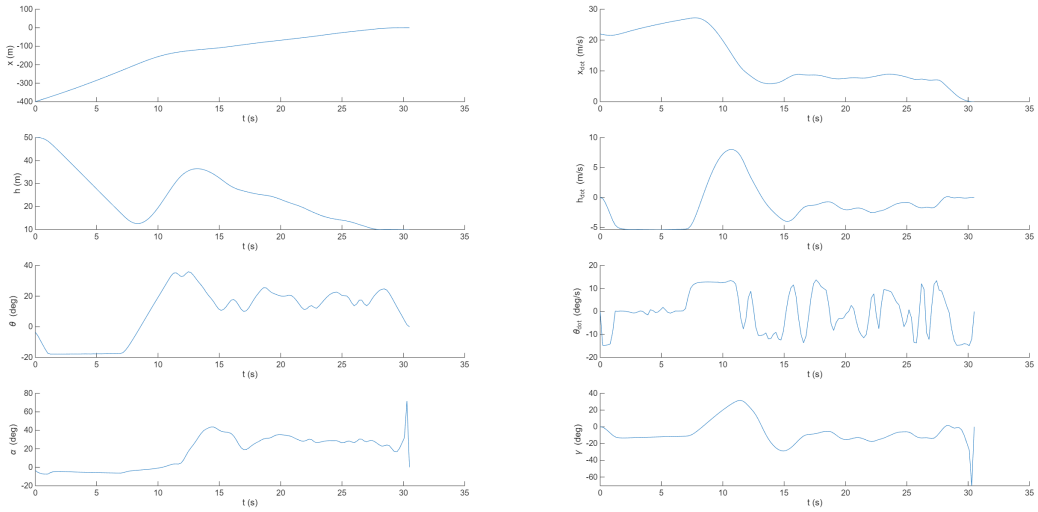


Figure 4.2: Trajectory Optimisation Solution States

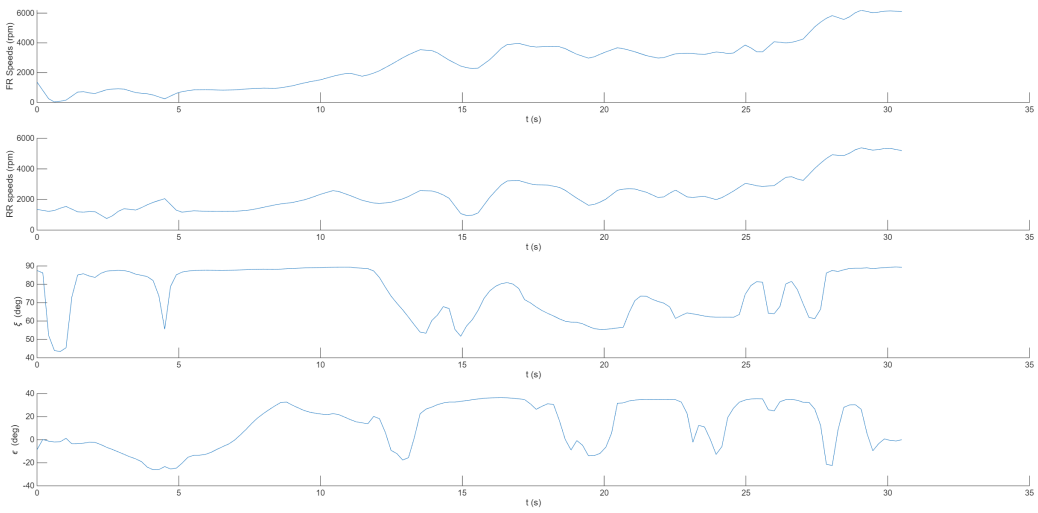


Figure 4.3: Trajectory Optimisation Solution Control Inputs

Another approach for analysing the trajectory is the energy states. To land the aircraft has two objectives, bleeding off the kinetic and potential energy. The solution trajectory immediately converts the potential energy to kinetic through the dive in the first 8s of the approach, followed by a pull up into a stall, that hits the pitch rate constraint causing the gain in height, however this stall also dissipates the vast majority of the kinetic energy, which remains largely constant in final stage of the approach (12s-30.5s) as the aircraft holds the upper angle of attack constraint (20 deg) and therefore a stall stabilised with the quad rotors to slowly approach the hover for handover.

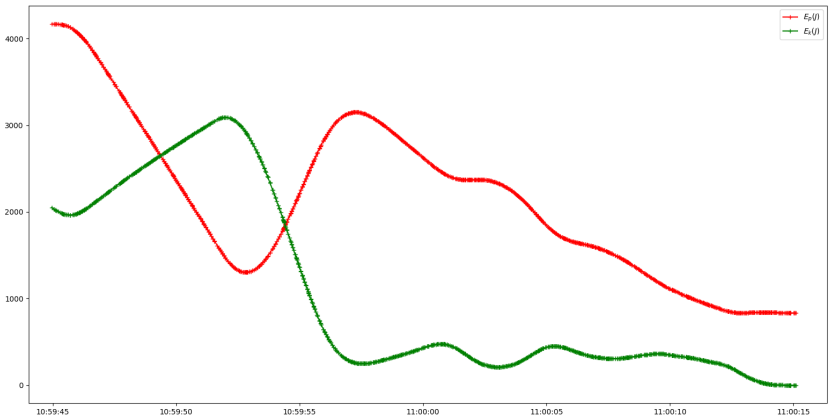


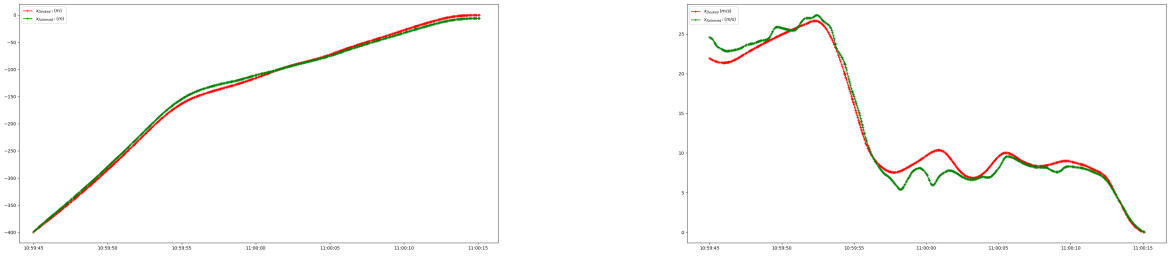
Figure 4.4: Landing Approach Energy States

4.2 Control Results

The longitudinal LQR controller prioritises tracking the positions over the associated velocities as seen in figures 4.5 through 4.7. This trade off was made on the basis of a small error in velocities would be less detrimental than a error in position, for example during the initial dive towards the ground as seen in first 8s of the landing approach as position overshoot could result in the aircraft nosediving into the ground especially if this controller was run with a trajectory that prioritised efficiency more and had the termination condition at ground level rather than a 10m offset. The aircraft pitch 4.7 does track far worse than the other two position in the LQR, that is due to the coupling of pitch to altitude and longitudinal position, and the controller prioritising the other two over pitch. This tradeoff was also made in the lateral controller as seen in the difference of tracking performance between 4.8 and 4.9, 4.10.

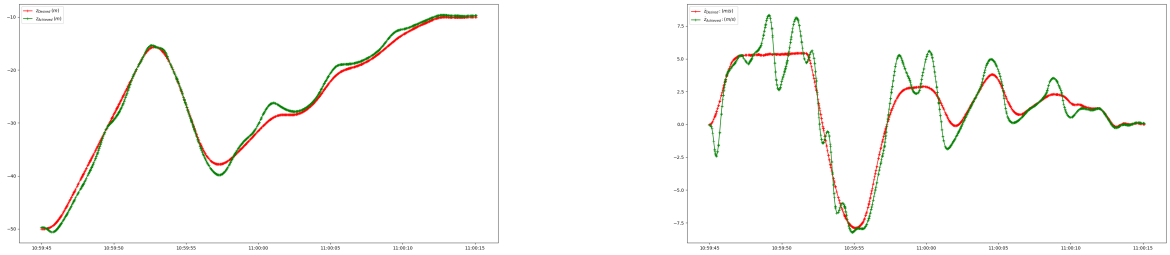
Further improvement could be made to the lateral control by incorporating the rudder and aileron into the script controller, this would also have the added benefit of relaxing the constraints on the waypoints in ArduPilot, as the current solution requires a waypoint that lies on the line drawn between the VTOL Landing and the previous waypoint to be the trajectory $x[0] - waypoint_{radius}$ from the landing waypoint. if it lies off the line the ArduPilot driven control surfaces, primarily the ailerons will drive the aircraft off track.

The other benefit of incorporating the full lateral dynamics would be that the lateral control could be achieved through the LQR which would provide a more optimal solution to the disturbance rejection control. Additionally if the lateral dynamics were integrated this controller would capable of driving any arbitrary trajectory, widening the possible applications.



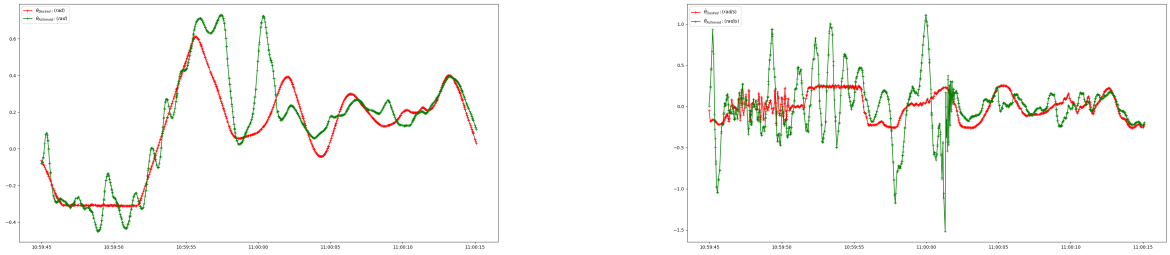
(a) x (b) $\dot{x}$

Figure 4.5: Simulation x and $\dot{x}$



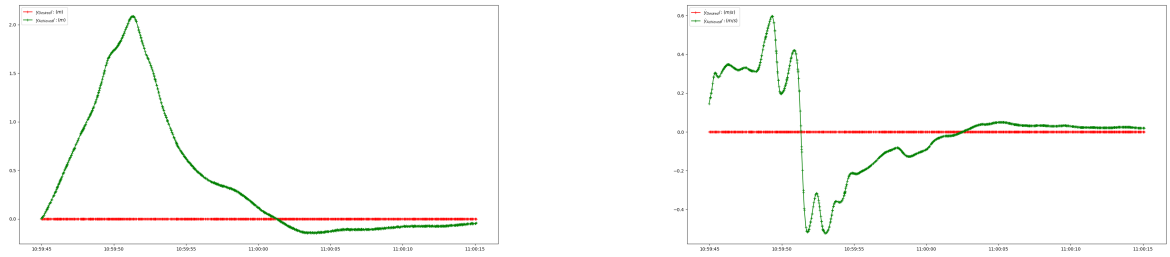
(a) z (b) $\dot{z}$

Figure 4.6: Simulation z and $\dot{z}$



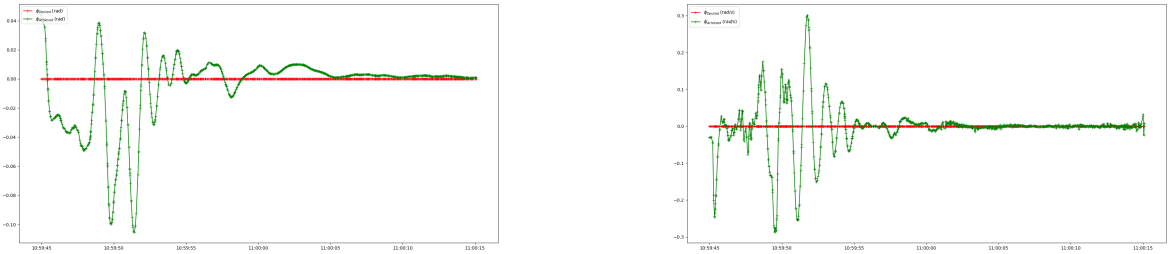
(a) θ (b) $\dot{\theta}$

Figure 4.7: Simulation θ and $\dot{\theta}$



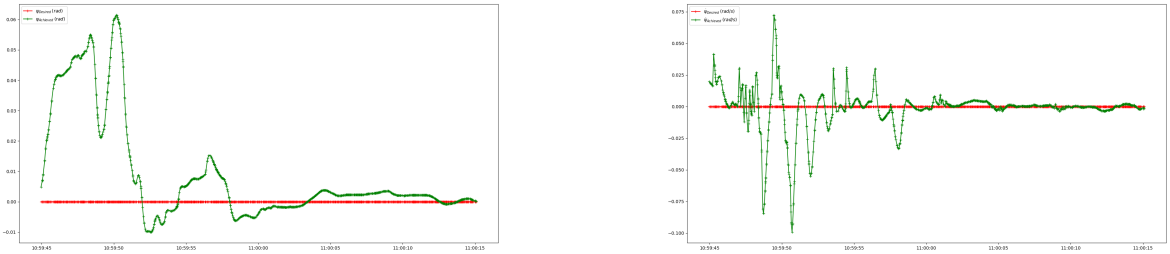
(a) y (b) $\dot{y}$

Figure 4.8: Simulation y and $\dot{y}$



(a) ϕ (b) $\dot{\phi}$

Figure 4.9: Simulation ϕ and $\dot{\phi}$



(a) ψ (b) $\dot{\psi}$

Figure 4.10: Simulation ψ and $\dot{\psi}$

The inputs throughout the trajectory (4.11-4.12), for the most part show that the controller is not overworking and oscillating the inputs excessively, with the exception of the elevator in the first third $(10:59:45 - 10:59:55)$ during the dive. This could likely be removed by increasing the cost on the elevator in the LQR.

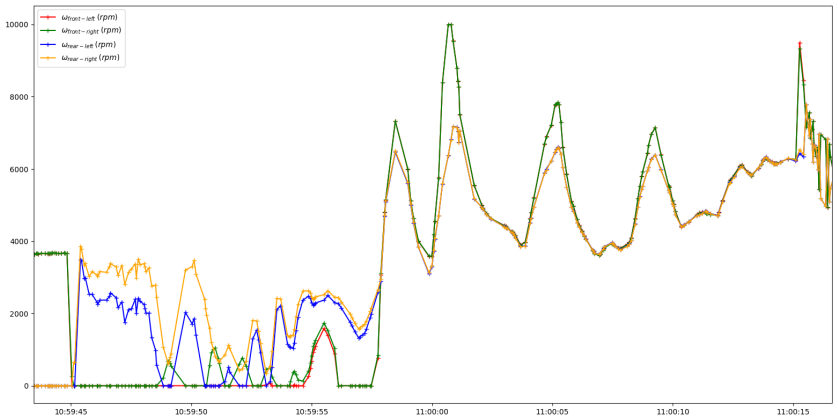
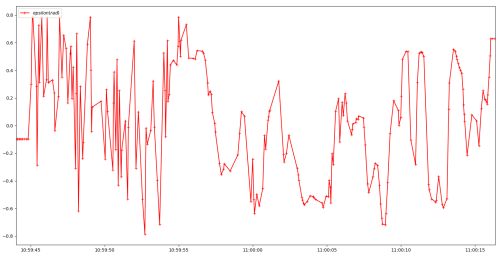
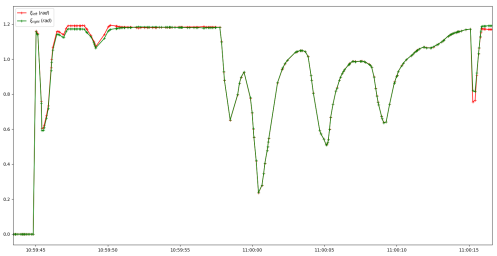


Figure 4.11: Simulation ω_{1-4}



(a) ϵ



(b) ξ

Figure 4.12: Simulation ϵ and ξ

The other aspect of evaluating the controller is disturbance rejection, figures 4.13, 4.14 show the controller's response to a step in wind from $0\,m/s$ to various speeds. The aircraft remains stable in all of these tests, although the controller especially in the headwind case sacrifices the aircraft attitude to stabilise the position compared to that of the ArduPilot controller.

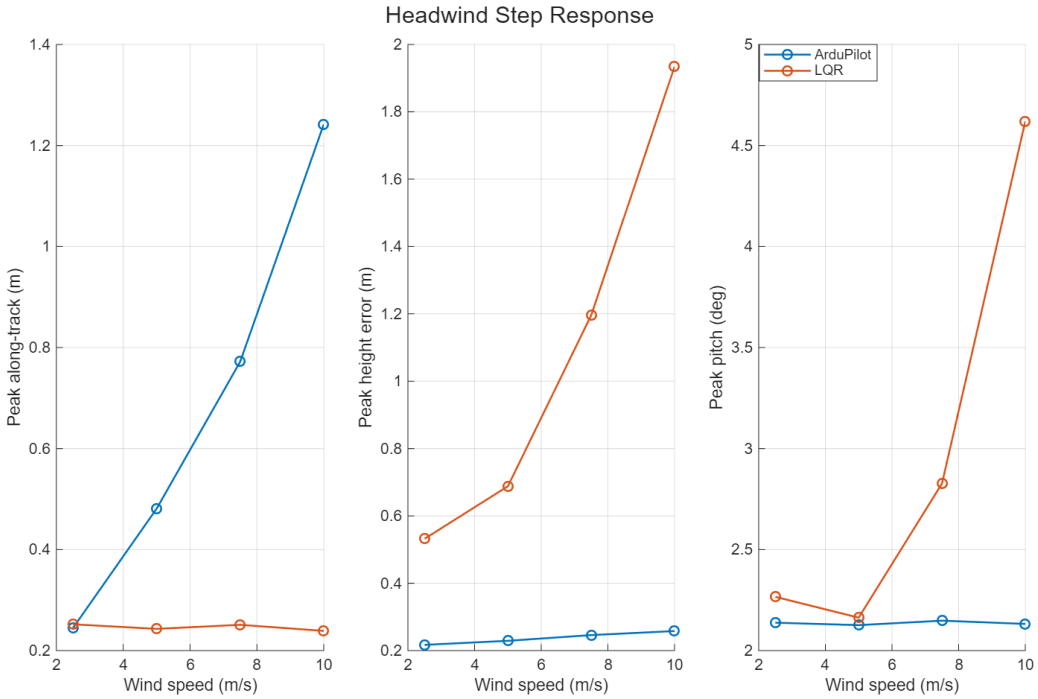


Figure 4.13: Headwind Step Response Peak in Hover

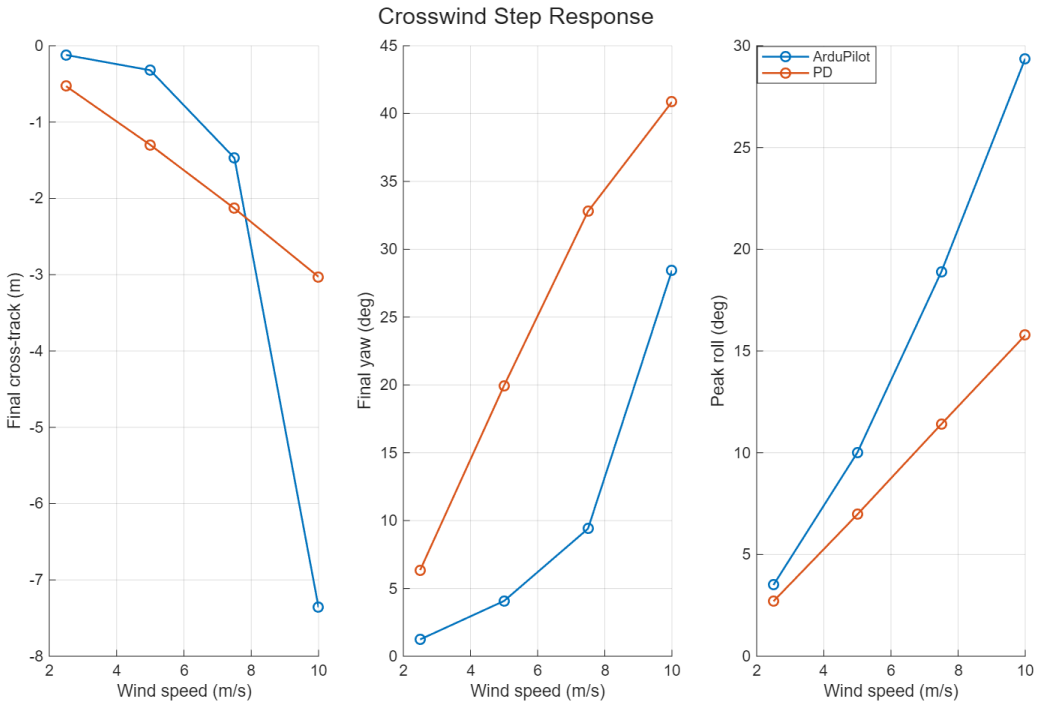


Figure 4.14: Crosswind Step Response Peak in Hover

Figure 4.2 shows that the primary reduction in energy consumption comes from the lack time spent in the hover phase (31.5s to 56s) as the Lua controller only hovers down 10m instead of the stock 50m, with the hover portion of the approach able to be reduced even further by changing the optimisation constraints in section 3.2 to achieve even higher efficiency, if the operator is comfortable with the increased risk.

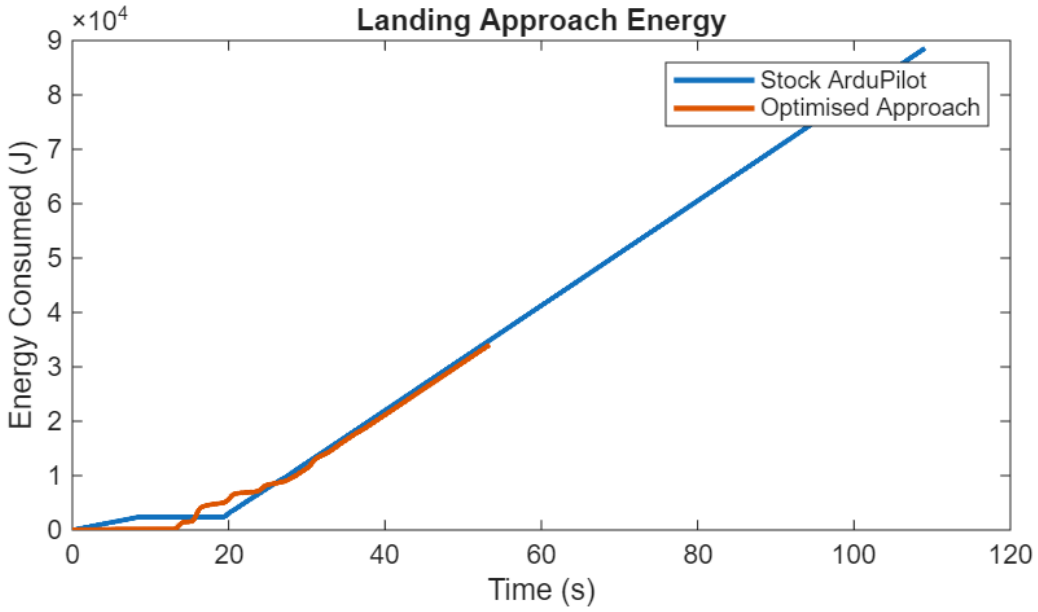


Figure 4.15: Landing Approach Energy Consumption

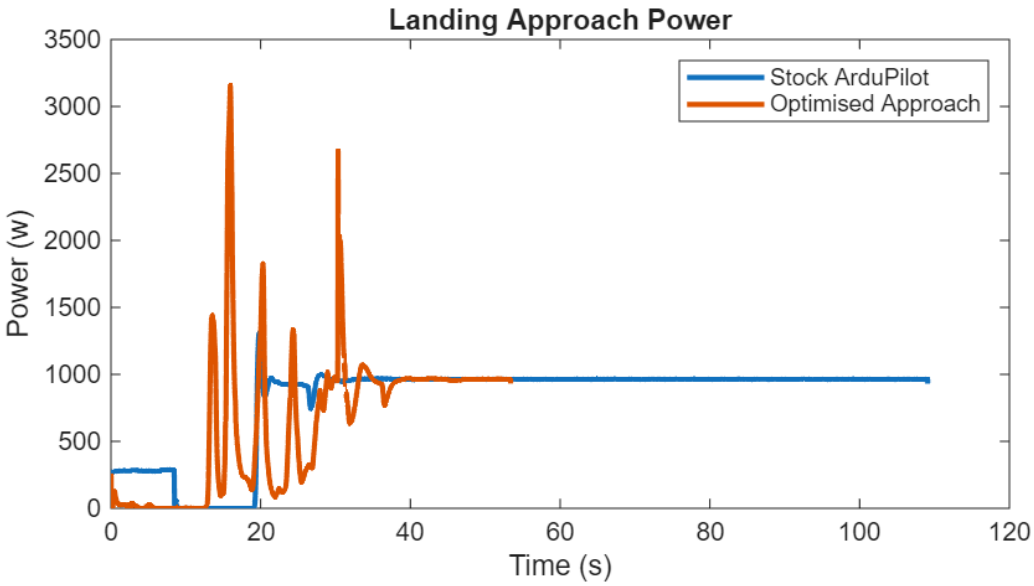


Figure 4.16: Landing Approach Power

4.2.1 Landing Rejection

As part of the implementation a landing rejection criteria was added, figure 4.17 shows the result of this being triggered and specifically the lack of controller wind up of the stock ArduPilot controller as it takes over control as the rejection is triggered.

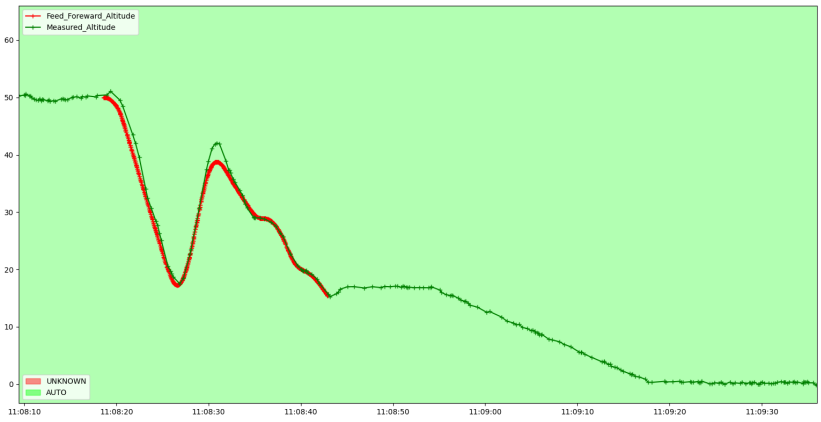


Figure 4.17: Altitude Floor Rejected Landing (15m)

Conclusions and Future Development

5.1 Conclusion

With the application of model based optimal control the energy consumption of a Arace Griffin Pro UAV during landing approach has been reduced by 61% and the duration reduced by 51% in simulation, This can improve efficiency, payload and range, without the need for any new hardware. Obviously as these are initial results more work needs to be done on this project to validate it before this could be pushed to a general ArduPilot build.

5.2 Future Development

The work put forward in this project while promising is not complete, and there are several areas of future work on this topic worth pursuing.

5.2.1 Validation

This project has only presented results from simulation. Real world flight testing is required to validate the control with the extra latency associated with the microcontroller, any model mismatch between the dynamical model presented in section 3.1 and real environmental disturbances such as wind, which have been simulated.

5.2.2 Model Expansion

While the initial decision was taken to limit the dynamical model to the longitudinal frame to reduce complexity, it now seems prudent to incorporate the lateral dynamics, as it would allow the lateral stabilisation to be incorporated into the LQR controller rather than the current PD solution, as well as allowing for three dimensional approaches, which would mitigate one of the limitations of the current trajectories requiring a long clear lane to approach the landing point.

5.2.3 Airframe types

This project in its current state exclusively supports tilt rotor powered lift UAVs, however, the same approach could be applied to tail sitters, tilt wings and quadplanes without excessive effort, allowing all powered lift aircraft to reap the benefits of this work.

5.2.4 System Identification

A dynamical model is required for both the trajectory optimisation and LQR control as shown in section 3, however to build this model a large number of parameters of the aircraft are required with

the most difficult to measure being the moments of inertia around the center of gravity. As such a system identification protocol would be a valuable addition to this work to provide users an estimate of these values especially if the model was extended to three dimensions.

Appendix A: The First

GitHub Repository with Trajectory Optimisation, Control Code, Simulation Set Up and Lua Script:
`https://github.com/lincoln-ka/Honours_Project`
GitHub Repository with Custom ArduPilot Build for new Rotor Model:
`https://github.com/lincoln-ka/ardupilot`

Bibliography

[1] M. Allenspach and G. J. J. Ducard. Nonlinear model predictive control and guidance for a propeller-tilting hybrid unmanned air vehicle. *Automatica*, 132:109790, 2021.

[2] B. D. O. Anderson and J. B. Moore. *Optimal Control: Linear Quadratic Methods*. Prentice Hall, 1990.

[3] Anthropic. Claude. `https://claude.ai`, 2026. Large language model. Accessed: 2026-11-06.

[4] ArduPilot Dev Team. Lua scripts. `https://ardupilot.org/plane/docs/common-lua-scripts.html`, . Accessed: 2026-10-12.

[5] ArduPilot Dev Team. Quadplane overview. `https://ardupilot.org/plane/docs/quadplane-overview.html`, . Accessed: 2026-10-12.

[6] ArduPilot Dev Team. SITL simulator (software in the loop). `https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html`, . Accessed: 2026-10-12.

[7] M. Bangura and R. Mahony. Thrust control for multirotor aerial vehicles. *IEEE Transactions on Robotics*, 33(2):390–405, 2017.

[8] J. T. Betts. Survey of numerical methods for trajectory optimization. *Journal of Guidance, Control, and Dynamics*, 21(2):193–207, 1998.

[9] A. E. Bryson and Y.-C. Ho. *Applied Optimal Control: Optimization, Estimation and Control*. Hemisphere, 1975.

[10] P. Cantrell. Settling with power. `https://www.copters.com/aero/settling.html`. Helicopter Aviation. Accessed: 2026-10-23.

[11] R. Cory and R. Tedrake. Experiments in fixed-wing UAV perching. In *AIAA Guidance, Navigation and Control Conference and Exhibit*, 2008.

[12] J. Deng, Y. Yang, J. Zhu, W. Liao, X. Yuan, and X. Wang. Pigeon-inspired transition trajectory optimization for tilt-rotor UAVs. *Drones*, 9(6):432, 2025. doi: 10.3390/drones9060432.

[13] G. J. J. Ducard and M. Allenspach. Review of designs and flight control techniques of hybrid and convertible VTOL UAVs. *Aerospace Science and Technology*, 118:107035, 2021.

[14] M. Faessler, A. Franchi, and D. Scaramuzza. Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories. *IEEE Robotics and Automation Letters*, 3(2):620–626, 2018.

[15] Y. Fu, W. Zhao, and L. Liu. Safe reinforcement learning for transition control of ducted-fan UAVs. *Drones*, 7(5):332, 2023. doi: 10.3390/drones7050332.

[16] C. R. Hargraves and S. W. Paris. Direct trajectory optimization using nonlinear programming and collocation. *Journal of Guidance, Control, and Dynamics*, 10(4):338–342, 1987.

[17] M. Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017. doi: 10.1137/16M1062569.

[18] M. P. Kelly. OptimTraj: Trajectory optimization for MATLAB, 2022. URL `https://github.com/MatthewPeterKelly/OptimTraj`.

[19] V. Klein and E. A. Morelli. *Aircraft System Identification: Theory and Practice*. AIAA, 2006.

[20] J. G. Leishman. *Principles of Helicopter Aerodynamics*. Cambridge University Press, 2nd edition, 2006.

[21] R. Mahony, V. Kumar, and P. Corke. Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor. *IEEE Robotics & Automation Magazine*, 19(3):20–32, 2012.

[22] P. Martin and E. Salaün. The true role of accelerometer feedback in quadrotor control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1623–1629, 2010.

[23] F. Miller. Todo: thesis title, 2025.

[24] J. Moore, R. Cory, and R. Tedrake. Robust post-stall perching with a simple fixed-wing glider using LQR-Trees. *Bioinspiration & Biomimetics*, 9(2):025013, 2014.

[25] R. Naldi and L. Marconi. Optimal transition maneuvers for a class of V/STOL aircraft. *Automatica*, 47(5):870–879, 2011.

[26] W. J. Rugh and J. S. Shamma. Research on gain scheduling. *Automatica*, 36(10):1401–1425, 2000.

[27] A. S. Saeed, A. B. Younes, C. Cai, and G. Cai. A survey of hybrid unmanned aerial vehicles. *Progress in Aerospace Sciences*, 98:91–105, 2018.

[28] R. Tedrake, I. R. Manchester, M. Tobenkin, and J. W. Roberts. LQR-trees: Feedback motion planning via sums-of-squares verification. *The International Journal of Robotics Research*, 29(8): 1038–1052, 2010.

[29] H. Zhao, B. Wang, Y. Shen, Y. Zhang, N. Li, and Z. Gao. Development of multimode flight transition strategy for tilt-rotor VTOL UAVs. *Drones*, 7(9):580, 2023. doi: 10.3390/drones7090580.